

线程池 OOM 的原因：

1. 任务队列无限增长(使用无界队列)。
2. 线程池中的线程数过多，系统无法处理。
3. 线程池中的任务存在内存泄漏。
4. 拒绝策略不当，导致内存溢出。

解决方案：

1. 使用合理的线程池参数(核心线程数和最大线程数)。
 2. 使用有限的任务队列(如 `ArrayBlockingQueue`)。
 3. 使用合适的拒绝策略(如 `CallerRunsPolicy`)。
 4. 监控线程池的运行状况。
 5. 确保任务执行过程中的内存管理。
 6. 及时关闭线程池，避免资源浪费。
-

JVM OOM 排查 — 如何定位并解决内存溢出问题

在 Java 程序运行时，`OutOfMemoryError` (OOM) 是一种常见的错误，表示 JVM 无法分配足够的内存资源。OOM 错误通常是由内存泄漏、堆内存不足、方法区内存溢出等原因引起的。排查 OOM 问题是每个 Java 开发人员必备的技能，以下是 JVM OOM 错误的排查方法和常见解决方案。

一、常见的 JVM OOM 类型

1. **Java heap space**: 堆内存溢出，通常是由于对象创建过多或内存泄漏导致的。
 2. **PermGen space** (Java 8 之前): 方法区溢出，通常是由于类加载过多、静态对象过多，导致 PermGen 空间不足。
 3. **Metaspace** (Java 8 及以后): Metaspace 取代了 PermGen，但同样可能因为类加载过多导致内存溢出。
 4. **Direct buffer memory**: 直接内存溢出，通常由于使用 NIO (`ByteBuffer.allocateDirect()`) 分配了过多的直接内存。
-

二、JVM OOM 排查步骤

1. 分析 OOM 错误信息

当发生 `OutOfMemoryError` 时, JVM 会输出错误信息, 帮助我们定位内存溢出的具体位置。错误信息的类型可以帮助我们确定是哪一块内存区域发生了溢出。

常见的 OOM 错误信息:

- `java.lang.OutOfMemoryError: Java heap space`
 - 说明堆内存不足, 可能是由于内存泄漏或大量对象创建。
- `java.lang.OutOfMemoryError: PermGen space` (Java 8 之前)
 - 说明 PermGen 区域内存不足, 通常是因为类加载过多。
- `java.lang.OutOfMemoryError: Metaspace` (Java 8 及以后)
 - 说明 Metaspace 区域内存不足, 通常是因为类加载过多。
- `java.lang.OutOfMemoryError: Direct buffer memory`
 - 说明直接内存不足, 通常是由于过多的 `ByteBuffer.allocateDirect()` 引起。

根据错误信息, 可以初步定位问题所在的内存区域。

2. 启用 JVM 内存分析参数

为了更好地理解 JVM 内存的使用情况, 可以启用一些参数, 来获取详细的内存分配和垃圾回收 (GC) 信息。

常见 JVM 参数:

- `-XX:+HeapDumpOnOutOfMemoryError`:
 - 启动时启用堆转储, 当发生 OOM 时生成 `.hprof` 文件, 用于堆内存分析。
 - 参数示例: `-XX:+HeapDumpOnOutOfMemoryError`
`-XX:HeapDumpPath=/path/to/dump.hprof`
- `-Xlog:gc*`:
 - 启用 GC 日志记录, 用于监控 GC 过程和内存分配。
 - 参数示例: `-Xlog:gc*`
- `-Xms` 和 `-Xmx`:
 - 设置 JVM 的初始堆大小和最大堆大小。
 - 示例: `-Xms512m -Xmx2g`
- `-XX:MaxMetaspaceSize`:
 - 设置 Metaspace 的最大大小。

- 示例: `-XX:MaxMetaspaceSize=512m`

3. 使用 `jmap` 和 `jstat` 工具分析堆内存

- `jmap`: 用于分析 Java 堆内存的工具, 可以查看堆内存使用情况并生成堆转储文件。

1. 查看堆内存使用情况:

使用 `jmap -heap <pid>` 命令可以查看当前进程的堆内存使用情况。

- 示例命令: `jmap -heap 1234`

2. 生成堆转储文件:

使用 `jmap -dump:format=b,file=<file-path> <pid>` 命令生成堆转储文件, 用于后续分析。

- 示例命令: `jmap -dump:format=b,file=heap_dump.hprof 1234`

4. 使用 `jstat` 工具分析垃圾回收

`jstat` 工具可以实时查看 JVM 的垃圾回收 (GC) 统计信息, 帮助我们判断是否频繁的 GC 导致了 OOM。

查看垃圾回收统计信息:

- 命令: `jstat -gc <pid>`
 - 示例输出:

S0C	S1C	S0U	S1U	EC	EU	OC	OU	MC	MU	CCSC
CCSU	YGC	YGCT	FGC	FGCT	GCT					
512.0	512.0	0.0	0.0	1024.0	0.0	2048.0	2048.0	512.0	512.0	128.0
128.0	10	0.101	2	0.02	0.12					

 - `S0U`、`S1U`: 表示 Survivor 区的使用情况。
 - `EU`: 表示 Eden 区的使用情况。
 - `YGC` 和 `FGC`: 分别表示年轻代 GC 和老年代 GC 次数, 可以帮助判断 GC 是否频繁。

ThreadLocal 内存泄漏问题

ThreadLocal 内存泄漏是因为 ThreadLocalMap 中 Entry 的 key 是弱引用, 而 value 是强引用。当 ThreadLocal 被回收后, Entry 的 key 会变成 null, 但 value 仍被 Thread 强引用。如果线程长期存活(如线程池), value 无法释放, 从而造成内存泄漏。解决方式是在使用完后必须调用 remove()。

key 是弱引用: 为了防止内存泄漏, 避免线程池中的线程复用时有对 ThreadLocal 的强引用, 使得 ThreadLocal 对象可以在没有任何强引用时被垃圾回收。

value 是强引用: 线程局部变量是线程的独立状态, 在整个线程生命周期内需要保持不变。使用强引用可以确保该线程的局部变量在任务执行期间始终有效, 不会被意外回收。

CAS 有什么问题？

主要问题有 **ABA** 问题、自旋导致 **CPU** 占用飙升, 以及 只能保证原子性, 无法保证顺序性。

解决方案: 通过引入版本号(如 AtomicStampedReference)、自旋次数控制等方法进行优化。

Q3-1: ABA 问题怎么解决？

使用版本号(AtomicStampedReference)或标记来确保 CAS 操作能感知值是否真正发生变化。

Q3-2: CAS 为什么会 CPU 飙高？

自旋锁导致在高并发场景下频繁自旋, 消耗大量 CPU 资源。可以通过限制自旋次数和自适应自旋来优化。

Q3-3: LongAdder 为什么性能高于 AtomicLong？

LongAdder 通过分段 Cell 来减少竞争, 避免了 AtomicLong 在高并发下的热点竞争, 性能明显更优。

ReentrantLock 底层原理？

依赖 AQS 来管理锁状态, 通过 CAS 和队列来实现锁的获取、释放及线程同步。

支持 公平锁和 非公平锁, 公平锁确保线程按请求顺序获取锁, 非公平锁允许线程抢占。

Q2-1: AQS 的 state 表示什么？

state 是用来表示同步器的状态, 在不同同步器(如 ReentrantLock、Semaphore)中有不同的意义。

Q2-2: CLH 队列为什么是双向链表？

双向链表可以高效访问前驱节点，避免单向链表的性能瓶颈，优化线程状态修复和取消线程处理。

Q2-3: 公平锁 vs 非公平锁实现差异？

非公平锁允许线程抢占锁，吞吐量较高，但可能导致线程饥饿；公平锁保证线程公平性，避免饥饿，但吞吐量较低。

Q2-4: Condition 和 Object.wait() 的区别？

`Condition` 提供了更多灵活性，可以用于多个条件变量，支持线程间更加细粒度的协调，而 `Object.wait()` 只能基于单一条件与锁进行同步。

synchronized 底层是怎么实现的？

synchronized 底层实现：基于 对象头(Mark Word) 和 锁状态管理，通过 **CAS** 和 **Monitor** 来实现线程同步，锁会经历 偏向锁 → 轻量级锁 → 重量级锁 的过程。

Q1-1: 对象头结构是什么？Mark Word 存什么？

对象头与 **Mark Word**: **Mark Word** 存储对象的元数据，包括锁的状态(偏向锁、轻量级锁、重量级锁)，它随着锁的状态变化而变化。

Q1-2: 锁升级过程为什么不可降级？

锁的升级不可降级：由于性能考虑和操作复杂性，**synchronized** 锁从轻量级锁升级为重量级锁后，不再降级。

Q1-3: 为什么轻量级锁要自旋？自旋次数如何控制？

轻量级锁自旋：自旋是为了减少线程阻塞带来的性能开销，适用于低竞争场景。自旋次数由 JVM 控制，并在竞争激烈时转为阻塞。

Q1-4: 什么情况下会直接膨胀成重量级锁？

膨胀成重量级锁的情况：当锁的竞争过于激烈，且自旋失败时，轻量级锁会膨胀成重量级锁，使用操作系统的互斥量来控制线程的调度。

fail-fast 机制

fail-fast 机制是一种在集合类发生结构变化时(如在遍历时删除或修改元素)立即抛出异常的机制。它用于检测并发修改或非法操作，避免程序在运行时出现不可预测的行为。

- 例如, `ArrayList` 和 `HashMap` 等集合类在遍历过程中, 如果发现集合被修改, 会抛出 `ConcurrentModificationException`。

fail-safe 机制

fail-safe 机制是指通过对集合类进行复制或其他方式, 确保在遍历过程中对集合的修改不会影响遍历过程。它通过创建集合的副本, 避免了并发修改的影响。

- 例如, `CopyOnWriteArrayList` 和 `ConcurrentHashMap` 等类采用了 fail-safe 机制, 允许并发访问但不抛出异常。

volatile

volatile 是 Java 中一个关键字, 表示某个变量的值在多个线程之间是共享的, 且每次读取都从主内存中获取, 而不是从线程的工作内存中获取。

- 主要功能:
 - 保证变量的 可见性: 当一个线程修改了 `volatile` 变量的值, 其他线程能够立刻看到这个变化。
 - 防止 指令重排: 禁止 JVM 对 `volatile` 变量的访问进行指令重排, 保证语义的正确性。
- 注意: `volatile` 只保证可见性和禁止指令重排, 不保证原子性。如果需要保证操作的原子性, 仍然需要使用 `synchronized` 或原子类。

线程池拒绝策略

当线程池中的线程数已达到最大值, 并且队列也满时, 任务会被拒绝执行。

`ThreadPoolExecutor` 提供了几种拒绝策略:

- **AbortPolicy**: 抛出 `RejectedExecutionException`。
- **CallerRunsPolicy**: 由提交任务的线程执行任务。
- **DiscardPolicy**: 丢弃任务, 不做任何处理。
- **DiscardOldestPolicy**: 丢弃队列中最旧的任务, 再尝试提交当前任务。

线程池 OOM

- **OOM (OutOfMemoryError)** 是指线程池中的任务过多, 导致内存不足。为了避免 OOM 错误, 可以通过:
 - 限制 **maximumPoolSize** 和 **workQueue** 的大小。
 - 配置合理的 **keepAliveTime**, 确保线程不会长时间占用内存。
 - 使用拒绝策略, 防止任务过多导致系统崩溃。

锁升级过程

锁升级是 JVM 中的优化策略, Java 中的锁从较轻的锁逐渐升级到较重的锁, 具体的过程如下:

1. 偏向锁 (Biased Locking):
 - 这是 Java 锁的最初状态。在没有竞争的情况下, JVM 偏向于将锁分配给第一个获得锁的线程, 这样就避免了其他线程争夺锁的开销。
 - 优势: 当线程长期占用锁时, 偏向锁避免了重复的锁获取操作, 减少了性能开销。
 - 升级条件: 当其他线程竞争锁时, 偏向锁会升级为轻量级锁。
2. 轻量级锁 (Lightweight Locking):
 - 当偏向锁无法使用时, JVM 会将锁升级为轻量级锁, 使用 **CAS** (Compare And Swap) 原子操作或 **Spinlock** 来尝试获得锁。
 - 轻量级锁是无阻塞的, 但在竞争激烈时会变成重量级锁。
3. 重量级锁 (Heavyweight Locking):
 - 如果有多个线程竞争同一个锁, 轻量级锁升级为重量级锁, 使用操作系统提供的互斥量 (如 **mutex**) 来保证同步。
 - 这时线程会被阻塞, 并且会消耗更多的 CPU 和内存资源。

公平锁 vs 非公平锁

- 公平锁: 线程按照请求的顺序 (先来先得) 获得锁。 **ReentrantLock** 支持公平锁, 通过构造函数的参数来指定。

```
ReentrantLock fairLock = new ReentrantLock(true); // 公平锁
```

 - 优点: 避免了线程饥饿现象。
 - 缺点: 性能开销较大。
- 非公平锁: 线程获取锁时, 不按照顺序排队, 而是随机选择一个等待中的线程来获取锁, 性能较高。

```
ReentrantLock unfairLock = new ReentrantLock(); // 默认非公平锁
```

 - 优点: 性能较好, 减少了上下文切换。
 - 缺点: 可能导致线程饥饿问题。

乐观锁 vs 悲观锁

- 悲观锁：认为并发冲突会频繁发生，因此每次访问共享资源时都会加锁，避免其他线程修改数据。
 - 实现：synchronized 和 ReentrantLock。
- 乐观锁：认为并发冲突不常发生，所以在访问共享资源时不加锁，而是进行数据验证，只有当数据发生冲突时才会采取其他措施。
 - 实现：通常通过 CAS 操作实现。

总结：

- 悲观锁：加锁操作，保证数据一致性，适合高并发的写操作。
- 乐观锁：不加锁，适合高并发的读操作。

注解原理

注解本质上是元数据，它们以 @ 符号开始，并且通常附加在类、方法、字段等代码元素上。注解本身不会改变程序的行为，但它们可以在编译、类加载、运行时通过工具或框架进行处理。

注解的工作原理：

- 编译时处理：编译器可以通过注解对代码进行处理。例如，Java 编译器通过注解生成额外的代码。
- 类加载时处理：一些框架（如 Spring、Hibernate）在类加载时通过反射读取注解，基于注解的信息做出相应的配置或行为。
- 运行时处理：某些注解会在程序运行时被读取，并由反射机制执行相关操作。

B+树的特征

B+树在 B树 的基础上做了一些改进，具有以下特征：

- 每个节点可以有多个子节点：B+树是一种多路查找树，节点可以有多个子节点（ m -叉树）。B+树的每个节点可以包含 $m-1$ 个键和 m 个指向子节点的指针。
- 所有叶子节点都在同一层：B+树是平衡树，所有的叶子节点都在同一层，这意味着从根节点到任何一个叶子节点的路径长度相同。这个特性保证了 B+树查询操作的效率。

- 叶子节点通过链表连接:所有叶子节点之间通过 双向链表 连接。这样, B+树不仅支持精确查询, 还能高效地进行范围查询(如 SQL 查询中的 **BETWEEN** 操作)。
- 内部节点仅存储索引:与 B树不同, B+树的 内部节点只包含索引值, 而不存储实际的数据。实际的数据仅存储在 叶子节点 中, 这使得 B+树的内部节点可以存储更多的索引, 提高了查找效率。
- 按顺序排列:B+树的叶子节点存储的记录按照键值从小到大的顺序排列, 且通过链表连接, 这使得范围查询非常高效。

慢查询优化

MySQL 支持慢查询日志功能, 用于记录执行时间较长的 SQL 查询。这有助于优化数据库性能, 找出瓶颈。

5.1 慢查询日志开启

- 可以通过设置 **long_query_time** 来指定查询执行时间超过多少秒的 SQL 被记录到慢查询日志中。
- 通过设置 **log_slow_queries** 启用慢查询日志。

5.2 慢查询优化策略

- 查询优化:通过分析慢查询日志, 找出执行时间较长的查询。然后通过增加索引、修改 SQL 语句、拆分复杂查询等方式进行优化。
- 使用 **EXPLAIN**:使用 **EXPLAIN** 语句分析 SQL 执行计划, 找出查询中的问题(如全表扫描、索引未使用等)。
- 索引优化:确保常用的查询条件字段有合适的索引, 以提高查询效率。

消息队列的解决的核心问题

2.1 削峰填谷

- 削峰填谷 是指通过消息队列来平滑流量波动, 避免高并发时系统压力过大。
- 场景:当系统面临短时间内的大量请求时, 消息队列可以将这些请求缓存起来, 系统以较平稳的速度处理。

实现方式:

- 在高峰期将请求暂时放入队列中, 慢慢处理。
- 在低谷期, 系统逐渐处理积压的请求, 避免系统崩溃。

2.2 解耦

消息队列的一个核心特性是 解耦。它通过将生产者和消费者解耦，避免了直接的依赖关系，使得应用组件之间的通信变得松耦合。

- 好处：
 - 生产者和消费者可以独立开发和部署。
 - 可以在不影响其他系统的情况下，独立升级系统的一部分。
- 例子：
 - Web 应用可以通过消息队列将用户请求放入队列中，而消费者系统(如日志处理系统、订单处理系统)可以独立处理这些请求。

2.3 异步化

消息队列通常用于 异步化 处理，意味着系统不需要等待某个操作完成就可以继续处理其他任务。

- 优势：
 - 提高系统吞吐量:不必等到某个任务完成，直接将任务交给消息队列，继续执行其他任务。
 - 降低系统响应时间:用户请求不必等待所有处理完成。
- 例子：
 - 在电商系统中，当用户下单时，支付、库存更新和发货可以异步处理。

消息队列的可靠性

3.1 消息可靠性

消息队列系统中的 消息可靠性 确保消息在发送、存储、消费的过程中不会丢失，并且能够正确地处理。

- 保障机制：
 - 消息持久化:将消息持久化到磁盘，以防止系统崩溃导致消息丢失(如 Kafka、RabbitMQ)。
 - 消息确认:消费者确认已成功消费消息。常见的确认机制有 自动确认 和 手动确认。
 - 重试机制:消息传递失败时，自动重试直到成功。
- 问题：
 - 消息丢失:系统崩溃、网络问题等可能导致消息丢失。
 - 消息重复消费:消费者消费相同的消息多次，可能会导致数据不一致。

3.2 消息顺序性

- 保证消息顺序性 是指在消费时，消息的顺序应该与发送时的顺序一致。
- 问题：

- 在高并发情况下, 分布式系统可能会打乱消息顺序, 特别是在多个消费者或多个分区的情况下。
- 解决方案:
 - 单分区: 保证顺序消费, 确保每个消息只由一个消费者消费。
 - 分区顺序: 根据消息的某些属性(如 ID、用户、订单等)将消息划分到不同的分区, 保证同一分区内的顺序。

3.3 幂等性

幂等性 是指消息处理的结果应该是相同的, 无论同一消息处理多少次。

- 问题:
 - 在网络波动或消费者重复处理的情况下, 可能会导致同一消息被消费多次。
- 解决方案:
 - 唯一标识: 为每条消息生成唯一标识符(如 UUID), 在消费时检查是否已经处理过该消息。
 - 数据库层面: 在数据库中使用唯一约束来保证同一操作不会重复执行。

高并发系统

设计的关键在于如何平衡系统的负载、保证性能、避免瓶颈。常见的设计模式包括:

- 削峰填谷: 平滑流量波动。
 - 消息队列: 使用消息队列(如 Kafka、RabbitMQ 等)将大量请求异步处理, 保证系统可以逐步消费请求, 而不会因为瞬时高峰造成系统崩溃。
 - 限流: 对请求数量进行限制, 通过控制请求速率来减少流量高峰期的压力。
 - 缓存: 将热点数据缓存起来, 减少高峰期的数据库访问压力。
- 限流: 防止过载。
 - 漏桶算法(**Leaky Bucket**): 请求流量有一个固定的出流速率, 超出限度的请求会被丢弃。适用于控制请求的速率。
 - 令牌桶算法(**Token Bucket**): 在固定时间内向桶中放置一定数量的令牌, 每个请求获取一个令牌, 没令牌的请求被拒绝。适用于平滑限流。
 - 计数器法: 通过记录单位时间内请求的数量来判断是否超过阈值, 如果超过, 则拒绝请求。
- 缓存: 提高响应速度。
 - 热点数据缓存: 将频繁访问的数据(如用户资料、商品信息等)存储在缓存中(如 Redis、Memcached), 减少数据库查询次数。
 - 缓存失效策略: 根据缓存的数据更新频率设置适当的过期时间。常见的策略有 定时过期、**LRU(Least Recently Used)** 缓存淘汰算法。
 - 双缓存策略: 避免缓存穿透和缓存击穿问题。例如, 使用双缓存来避免读取缓存时直接访问数据库。
- 异步处理: 提高系统吞吐量。

- 消息队列:将需要异步处理的任务(如订单处理、邮件发送)放入消息队列, 后台服务异步消费消息, 减少系统的同步等待时间。
 - 线程池:使用线程池来执行异步任务, 将耗时操作放到后台线程执行, 避免主线程阻塞。
 - 回调机制:在异步任务完成后, 触发回调函数通知客户端或其他系统模块。
 - 解耦:减少模块间的直接依赖。
 - 消息队列:通过消息队列将服务之间的调用解耦。例如, 生产者将消息发布到队列中, 消费者从队列中获取并处理, 避免直接调用。
 - 事件驱动:通过事件机制通知系统的变化, 而不是直接调用系统方法。常见的技术栈如 **EventBus**、**Kafka**。
 - 隔离:防止单个模块故障影响全局。
 - 服务隔离:通过微服务架构将不同的业务功能拆分成独立的服务, 每个服务负责不同的任务, 减少单一服务的负载。
 - 数据库隔离:不同模块或应用可以使用不同的数据库实例, 避免互相影响。
 - 线程池隔离:对不同的业务逻辑使用不同的线程池, 避免一个业务的高并发请求影响其他业务。
 - 分片:提高存储和计算的扩展性。
 - 数据库分片:根据某些规则(如哈希、范围等)将数据分配到多个数据库或表中, 以实现横向扩展。
 - 任务分片:将大规模任务(如大数据处理、批量任务)拆分为多个小任务并分发到多个工作节点处理。
 - 负载均衡:分散请求压力, 提升可用性。
 - **DNS** 负载均衡:通过 DNS 将请求路由到不同的服务器或集群节点。
 - 反向代理负载均衡:使用反向代理服务器(如 Nginx、HAProxy)来将请求按策略分发到后端的服务器池。
 - 应用层负载均衡:通过应用层的负载均衡算法(如轮询、加权轮询、最少连接等)分发请求。
-

Kafka高可用

Kafka 通过分区多副本机制、Leader-Follower 模型、分区重分配、自动故障转移、幂等性设计等手段, 确保了其在生产环境中的高可用性和容错性。在实际应用中, Kafka 的高可用性还需要依赖 **ZooKeeper** 集群的稳定性以及合理的配置, 如副本因子、分区数、Producer 和 Consumer 的设置等。通过合理的设计和配置, Kafka 可以在面对单点故障或网络分区时依然提供稳定的服务。

JDK7 vs JDK8 ConcurrentHashMap 区别

JDK7 的 **ConcurrentHashMap**:

- 使用 分段锁(**Segment Lock**):整个 **ConcurrentHashMap** 被划分为多个段, 每个段拥有一个锁。这样可以提高并发性, 但锁的粒度较粗。

JDK8 的 **ConcurrentHashMap**:

- 不再使用 分段锁, 改为 桶级别的锁。
- 采用 **synchronized** 和 **CAS** 的结合, 提供更细粒度的锁定。
- 引入了 树化(**TreeBins**):当桶中的链表长度超过一定阈值(默认为 8), 会转换为红黑树, 提升查询效率。
- 支持 并行计算:通过 **forEach**、**reduce**、**search** 等方法, 能在多核处理器上并行执行操作。

总结区别:

- JDK8 提供了更细粒度的锁管理, 减少了锁的竞争, 提高了并发性。
- 引入了树化技术, 提升了高负载情况下的性能。

高并发下如何避免锁表?

在高并发的场景下, 表锁会严重影响性能, 导致系统瓶颈。因此, 避免锁表是非常重要的。

避免锁表的策略:

1. 使用行级锁:优先使用行级锁而不是表级锁, 行级锁可以减少锁的粒度, 提高并发性。
 2. 分批次处理:对于大批量的写入操作, 分批次进行, 避免一次性锁住大量行。
 3. 减少锁的持有时间:优化 SQL 和事务, 尽量减少锁的持有时间, 避免长时间占用锁。
 4. 避免全表扫描:通过索引优化查询, 减少全表扫描的情况, 避免锁定整个表。
 5. 乐观锁:采用乐观锁机制, 通过版本号或时间戳控制并发, 避免锁竞争。
 6. 分区表:使用分区表将数据分割成多个逻辑表, 减少表的大小, 降低锁竞争的概率。
-
-
-
-
-
